

Cardeno, Faye L.
BSCS251A

This document summarizes the refactoring improvements applied to the project. The changes focus on SOLID principles, object-oriented design, maintainability, security, and separation of concerns.

1. Improving SRP: Separating Validation from Service

TaskService was responsible for both business operations and validation. Validation logic was extracted into a dedicated TaskValidator class, allowing the service to focus on orchestration while the validator handles input validation.

Benefit: Improved readability, maintainability, and testability.

Before:

```
public class TaskService {  
  
    private final TaskRepository repo;  
    private final List<NotificationStrategy> notifiers;  
  
    public Task createTask(TaskDTO dto) {  
        if (dto == null) throw new IllegalArgumentException("Task cannot be null");  
        if (dto.getTitle() == null || dto.getTitle().trim().isEmpty()) {  
            throw new IllegalArgumentException("Task title is required");  
        }  
  
        Task task = new Task(null, dto.getTitle(), dto.getDueDate(), false);  
        Task savedTask = repo.save(task);  
  
        notifiers.forEach(n -> n.send(savedTask));  
  
        return savedTask;  
    }  
}
```

After:

```
public Task createTask(TaskDTO dto) { 5 usages new *  
  
    // DRY: Validation logic is reused via TaskValidator  
    // SOLID: SRP - Delegation of validation to a specialized component  
    validator.validate(dto);
```

```

package com.grasp.cardeno.validation;

import com.grasp.cardeno.dto.TaskDTO;
import org.springframework.stereotype.Component;

@Component new *
public class TaskValidator {
    // SOLID: SRP - Handles only validation logic, keeping the Service clean.
    public void validate(TaskDTO dto) { 3 usages new *
        if (dto == null) throw new IllegalArgumentException("Task cannot be null");
        if (dto.getTitle() == null || dto.getTitle().trim().isEmpty()) {
            throw new IllegalArgumentException("Task title is required");
        }
    }
}

```

2. Improving DIP: Strengthening Abstraction

The service was tightly coupled to validation logic. By creating TaskValidator, the service now relies on abstractions and focuses only on coordinating application behavior.

Benefit: Reduced coupling and improved modularity.

Before:

```

@Service
@RequiredArgsConstructor
public class TaskService {

    private final TaskRepository repo;
    private final List<NotificationStrategy> notifiers;

    public Task createTask(TaskDTO dto) {
        if (dto == null) throw new IllegalArgumentException("Task cannot be null");
        if (dto.getTitle() == null || dto.getTitle().trim().isEmpty()) {
            throw new IllegalArgumentException("Task title is required");
        }

        Task task = new Task(null, dto.getTitle(), dto.getDueDate(), false);
        Task savedTask = repo.save(task);

        notifiers.forEach(n -> n.send(savedTask));

        return savedTask;
    }
}

```

After:

```
@Service new *
@RequiredArgsConstructor
public class TaskService {

    // DIP: Depends on Abstraction (NotificationStrategy) instead of concrete classes
    // GRASP: Low Coupling - Service is not aware of how notifications are sent
    private final TaskRepository repo;
    private final List<NotificationStrategy> notifiers;
    private final TaskValidator validator;

    // SOLID: SRP - Service is now only responsible for orchestration
    public Task createTask(TaskDTO dto) { // Susages new *

        // DRY: Validation logic is reused via TaskValidator
        // SOLID: SRP - Delegation of validation to a specialized component
        validator.validate(dto);

        // GRASP: Creator - Service is responsible for instantiating the Task entity
        Task task = new Task(id: null, dto.getTitle(), dto.getDueDate(), isCompleted: false);
        Task savedTask = repo.save(task);

        // GRASP: Polymorphism - Executes different notification behaviors via the same interface
        // SOLID: OCP - System is open for extension (new notifiers) but closed for modification
        notifiers.forEach( NotificationStrategy n -> n.send(savedTask));

        return savedTask;
    }
}
```

3. Improving Data Protection: API/DTO Separation

The controller previously returned the Task entity directly. A TaskResponse DTO was then created to expose only the data intended for clients.

Benefit: Better encapsulation, security, and API flexibility.

Before:

```
@RestController
@RequestMapping("/tasks")
public class TaskController {

    private final TaskService service;

    public TaskController(TaskService service) { this.service = service; }

    @PostMapping
    public ResponseEntity<Task> create(@RequestBody TaskDTO dto) {
        // Controller acts as the entry point (GRASP: Controller)
        Task createdTask = service.createTask(dto);
        return ResponseEntity.ok(createdTask);
    }
}
```

After:

```
@RestController new *
@RequestMapping("/tasks")
public class TaskController {
    private final TaskService service; 2 usages

    public TaskController(TaskService service) { this.service = service; } no usages new *

    @PostMapping new *
    public ResponseEntity<TaskResponse> create(@RequestBody TaskDTO dto) {
        // 1. Delegate business logic to service
        Task savedTask = service.createTask(dto);

        // 2. Transform the Entity into a Response DTO (Hiding internal database structure)
        TaskResponse response = new TaskResponse(savedTask.getId(), savedTask.getTitle());

        return ResponseEntity.ok(response);
    }
}
```

4. Improving Robustness: Centralized Exception Handling

Error handling was centralized using `GlobalExceptionHandler`, eliminating duplicated exception handling logic across controllers and services.

Benefit: Consistent error responses and easier maintenance.

Before:

Each service or controller method would have to manually try-catch errors, leading to duplicate code and inconsistent error formats.

After:

```
@ControllerAdvice new *
public class GlobalExceptionHandler {

    // Handles your existing TaskNotFoundException
    @ExceptionHandler(TaskNotFoundException.class) 1 usage new *
    public ResponseEntity<String> handleNotFound(TaskNotFoundException ex) {
        return new ResponseEntity<> (
            ex.getMessage(),
            HttpStatus.NOT_FOUND
        );
    }

    // Handles validation errors
    @ExceptionHandler(IllegalArgumentException.class) 1 usage new *
    public ResponseEntity<String> handleValidation(IllegalArgumentException ex) {
        return new ResponseEntity<> (
            ex.getMessage(),
            HttpStatus.BAD_REQUEST
        );
    }

    // Handles unexpected server errors
    @ExceptionHandler(Exception.class) 1 usage new *
    public ResponseEntity<String> handleGeneralException(Exception ex) {
        return new ResponseEntity<> (
            body: "An internal error occurred: " + ex.getMessage(),
            HttpStatus.INTERNAL_SERVER_ERROR
        );
    }
}
```

5. Improving Encapsulation: Protecting Domain State

The `@Setter` annotation was removed from the `Task` entity to prevent unrestricted modification of object state.

Benefit: Stronger encapsulation and improved domain integrity.

Before:

```
@Entity
@Getter @Setter
@NoArgsConstructor
@AllArgsConstructor
public class Task {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private LocalDate deadline;
    private boolean isCompleted;

    public boolean isOverdue() {
        return !isCompleted && deadline.isBefore(LocalDate.now());
    }
}
```

After:

```
@Entity new *
@Getter // OOP: Immutability - Removing @Setter protects object state integrity
@NoArgsConstructor
@AllArgsConstructor
// GRASP: High Cohesion - Task only manages task data
public class Task {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private LocalDate deadline;
    private boolean isCompleted;

    // GRASP: Information Expert - Task calculates its own status
    public boolean isOverdue() { 5 usages new *
        return deadline != null && !isCompleted && deadline.isBefore(LocalDate.now());
    }
}
```